MUSE-VFL: Multi-party Unified System for Private and Communication Efficient Backpropagation in Vertical Federated Learning

Ivan Tjuawinata*, Yann Fraboni†, Ziyao Liu*, Jun Zhao*, Pu Duan†, Kwok-Yan Lam*
* Nanyang Technological University
† Ant International

***Abstract*—Vertical federated learning (VFL) enables a cohort of parties with vertically partitioned data to collaboratively train a machine learning (ML) model without requiring them to centralise their data. Each party feeds its data to its local model, with output fed to a global model. However, this configuration requires parties to share some intermediary results during training, which include the output and the gradients of the local models. These intermediary results can reveal insights into the parties' data, and can be protected by secret sharing them with secure multiparty computation (MPC). However, this increases the total number of communications and makes the VFL training significantly slower. In this work, we introduce MUSE-VFL to accelerate the computation of the local gradients by using homomorphic encryption on top of MPC for parties to directly complete this computation during backpropagation. We show theoretically that MUSE-VFL improves the complexity of the MPC baseline. Our experiments, conducted on four different ML tasks, show that the runtime needed to compute the gradients of the local models significantly outweighs the combined runtime of all other steps. This highlights the significance of MUSE-VFL, with experiments demonstrating a training runtime faster by 30% to 35% for LAN and 32% to 50% for WAN.**

***Index Terms*—vertical federated learning, backpropagation, multi-party computation, homomorphic encryption.**

I. Introduction

Collaborative analysis is an essential method for multiple parties to uncover deeper insights in their data, with applications spanning various industries including finance [1, 2] and healthcare [3]. In an ideal world, the distributed data could be centralized by a trusted party that would then execute the data analysis agreed upon by the other parties. However, this solution requires parties to trust that this central party will act honestly and will not use the centralized data for other purposes. In addition, in the real world, centralizing the data is a challenging step due to the sensitive nature of the data and data privacy laws in various countries that may render such an approach unlawful, particularly when the data is distributed across different countries.

Vertical federated learning (VFL) [4–9] was introduced to train machine learning (ML) models on distributed data and to address these problems. Instead of centralizing the data and the training of the ML model, each party keeps its data on its premises and owns a local model to do a part of the training. With VFL, a global party centralises the outputs of the parties' model and trains a global model on them. VFL orchestrates the joint training of the local and global models. However, while with VFL the data is not centralised, information about the parties' data is still exposed through the outputs of the parties' models, which are computed on the parties' data and can be used to learn information regarding it. Therefore, in this work, we define VFL as private when the global party cannot access results computed on the local outputs and when every party can only access the gradient necessary to update its local model.

To satisfy our privacy definition, VFL needs to be combined with privacy-enhancing techniques, including secure multiparty computation (MPC) [7, 10, 11], homomorphic encryption (HE) [12, 13], differential privacy (DP) [14, 15], or a combination of them [16]. DP can be used to perturb the outputs of the local models with a well-calibrated noise to obfuscate the sensitive information in them and, therefore, in any computation made on these outputs. However, due to this perturbation, VFL with DP can lead to a model with lower performance. For this reason, in the rest of this work, we focus on MPC- and HE-based techniques that do not change the performance of the model trained with VFL. With MPC or HE, each party can use either secret-sharing or encryption to protect the outputs of the local models and the computations done on top of them in the rest of the training. However, a direct application of MPC and HE in providing privacy-enhanced training typically come with higher complexity and using these techniques slows down the VFL training significantly. Hence, the need to propose a communication-efficient solution for privacy-enhanced VFL.

To the best of our knowledge, no previous work considers directly the problem of privacy-enhanced VFL and proposes a solution satisfying our privacy considerations. Therefore, in this work, we first formulate VFL as an MPC problem to create a baseline solution in which

every party secret shares the output of its model with a set of computation servers to enable secure computation over sensitive data. However, to privately compute the gradient during backpropagation, every party also needs to secret share the partial derivatives of their outputs with respect to their parameters, which incurs a large amount of additional communications between the parties and the servers. We propose our novel solution, Multi-party Unified System for private and communication-Efficient backpropagation in VFL (MUSE-VFL), that uses HE on top of MPC to avoid this secret sharing by having instead each party homomorphically complete the gradient computation. We compare theoretically and experimentally MUSE-VFL to the MPC baseline. We show theoretically that our solution lowers the complexity of all the parties and computation servers, and experimentally that MUSE-VFL outperforms the MPC baseline by 30% to 35% for LAN and by 32% to 50% for WAN on four different ML tasks. We further observe that the percentage improvement is consistent across network configurations, showing it is not a fixed benefit.

This paper is structured as follows. In Section II, we introduce our notations, VFL, MPC, and HE. In Section III, we introduce the MPC baseline and MUSE-VFL. In Section IV, we compare the complexity of MUSE-VFL to that of the MPC baseline. Finally, in Section V, we perform this comparison experimentally.

II. Preliminaries

Before introducing MUSE-VFL, we present VFL in Section II-A, MPC in Section II-B, HE in Section II-C, how to homomorphically encrypt a secret shared value in Section II-D, and previous works on privacy-enhanced VFL in Section II-E.

A. Vertical Federated Learning

Vertical federated learning (VFL) is a variant of federated learning where the data is vertically aligned. Algorithm 1 provides the instructions for VFL to optimize the parameters of the model of each party when the participants share their intermediary results in plaintext. We detail each VFL step in this section and introduce the sets that need to be transmitted between parties to achieve VFL.

Let there be an *active* party $\mathcal{P}_0$ and *n passive* parties $\mathcal{P}_1, \cdots, \mathcal{P}_n$. We consider a dataset of K data samples denoted as $\mathcal{D} = \{x_k\}_{k=1}^K$, with $x_k = \{x_{1,k}, ..., x_{n,k}, y_k\}$. The dataset is distributed across the parties, where each passive party $\mathcal{P}_i$ holds the data slice $x_{i,k}$, and the active party $\mathcal{P}_0$ holds the label y_k. Each passive party owns a local model f_i with parameters θ_i. We define the set of feature representations $F_i[\mathcal{B}]$ that contains the feature representation of each data slice $x_{i,k}$ in the batch $\mathcal{B}$, i.e.

$$F_i[\mathcal{B}] = \{f_i(x_{i,k}) : x_k \in \mathcal{B}\}. \tag{1}$$

Algorithm 1 Vertical Federated Learning on Plaintext

Require: vertically split dataset $\mathcal{D}$, learning rate η, number of SGD steps T, batch size B.
1: Random initialization of $\theta_0, \cdots, \theta_n$.
2: **for** $t = 1, \cdots, T$ **do**
3: Sample a batch $\mathcal{B} \subseteq \mathcal{D}$ of size B.
4: $\mathcal{P}_i$ computes $F_i[\mathcal{B}]$, eq. (1).
5: $\mathcal{P}_i$ transmits $F_i[\mathcal{B}]$ to $\mathcal{P}_0$.
6: $\mathcal{P}_0$ performs a forward pass, eq. (2).
7: $\mathcal{P}_0$ computes the batch loss $\mathcal{L}(\mathcal{B})$.
8: $\mathcal{P}_0$ computes $\nabla_{\theta_0}\mathcal{L}(\mathcal{B})$.
9: $\mathcal{P}_0$ computes $U_i[\mathcal{B}]$, eq. (5).
10: $\mathcal{P}_i$ computes $L_i[\mathcal{B}]$, eq. (6).
11: $\mathcal{P}_0$ transmits $U_i[\mathcal{B}]$ to $\mathcal{P}_i$.
12: $\mathcal{P}_i$ computes the local gradient $\nabla_{\theta_i}\mathcal{L}(\mathcal{B})$.
13: $\mathcal{P}_i$ updates θ_i, eq. (3).
14: **end for**
Output: $\mathcal{P}_i$ gets θ_i.
Note: Alternatively to lines 11-12, $\mathcal{P}_i$ can transmit $L_i[\mathcal{B}]$ to $\mathcal{P}_0$, which computes then the local gradients.

Each passive party computes this set and transmits it to the active party. The active party owns a global model f_0 with parameters θ_0 that takes as input the output of each active party's local model, i.e., for a data sample x_k,

$$\hat{y}(x_k) = g(f_1(x_{1,k}, \theta_1), ..., f_n(x_{n,k}, \theta_n), \theta_0). \tag{2}$$

We consider the loss function $l(x_k)$, which measures the discrepancy between the model's output $\hat{y}(x_k)$ and the true label y_k for one data sample. For simplicity, we omit $\theta_0, \cdots \theta_n$ as arguments of l and refer to $l(x_k)$ instead of $l(x_k, \theta_0, \ldots, \theta_n)$. The total loss function is defined as $\mathcal{L}(\mathcal{D}) = \frac{1}{|\mathcal{D}|}\sum_{x\in\mathcal{D}} l(x)$, and the batch loss function as $\mathcal{L}(\mathcal{B}) = \frac{1}{|\mathcal{B}|}\sum_{x\in\mathcal{B}} l(x)$. At every optimization round, every party computes the gradient of the loss with respect to their parameters $\nabla_{\theta_i}\mathcal{L}(\mathcal{B})$ to update their parameters with a stochastic gradient descent (SGD) step

$$\theta_i \leftarrow \theta_i - \eta\nabla_{\theta_i}\mathcal{L}(\mathcal{B}). \tag{3}$$

The batch loss function is a linear combination of the loss function of each data sample in the batch. Hence, considering the linear property of differentiation, the gradient of the batch loss function satisfies $\nabla_{\theta_i}\mathcal{L}(\mathcal{B}) = \frac{1}{|\mathcal{B}|}\sum_{x\in\mathcal{B}} \nabla_{\theta_i} l(x)$. For a single data sample x_k, the gradient can be expressed with the chain rule as

$$\nabla_{\theta_i} l(x_{i,k}) = \frac{\partial l(x_{i,k})}{\partial f_i(x_{i,k}, \theta_i)} \cdot \frac{\partial f_i(x_{i,k}, \theta_i)}{\partial \theta_i}. \tag{4}$$

We respectively refer to *upper* and *lower* partial derivatives for the first and second term of equation (4). The first term is the partial derivative of the loss function with respect to the output of $\mathcal{P}_i$ shared by $\mathcal{P}_0$, and equals 1 for the active party. The second term is the derivative of the

model output with respect to its parameters, computed with backpropagation. We define the set of upper partial loss derivatives $U_i[\mathcal{B}]$ that the active party $\mathcal{P}_0$ must compute, i.e.

$$U_i[\mathcal{B}] = \left\{ \frac{\partial l(x_{i,k})}{\partial f_i(x_{i,k}, \theta_i)} : x_k \in \mathcal{B} \right\}, \tag{5}$$

and the set of lower partial derivatives $L_i[\mathcal{B}]$ that the passive party $\mathcal{P}_i$ must compute, i.e.

$$L_i[\mathcal{B}] = \left\{ \frac{\partial f_i(x_{i,k}, \theta_i)}{\partial \theta_i} : x_k \in \mathcal{B} \right\}. \tag{6}$$

The gradient can either be computed by the passive party $\mathcal{P}_i$ or the active party $\mathcal{P}_0$ and will require the other party to send its upper or lower partial derivatives respectively.

B. Multiparty Computation

A secret sharing scheme-based multiparty computation scheme (MPC) enables mutually distrusting parties to perform computation of a previously agreed function on their private input without the need to reveal information that cannot be inferred from the function output. A more complete discussion about secret sharing based MPC can be found in [17].

In this work, we consider the underlying secret sharing scheme to be done with a set of J computation servers $\mathcal{S}_1, \cdots, \mathcal{S}_J$. For a value v in the finite field $\mathbb{F}$, we define by $[\![v]\!]$ the additive secret sharing of v to each computation server and by $[\![v]\!]_j$ the secret share owned by the computation server $\mathcal{S}_j$. A secret value is secretly shared by having each server hold a random value that adds up to the secret value. For the purpose of this work, we assume that the underlying secret-sharing based MPC, which we denote by $\mathcal{F}_{MPC}$, has the following functionalities.

- $[\![v]\!] \leftarrow \mathcal{F}_{MPC}.\texttt{Share}(v)$: To secret share a value v with all the computation servers. Each server $\mathcal{S}_j$ receives the secret share $[\![v]\!]_j$.
- $v \leftarrow \mathcal{F}_{MPC}.\texttt{Reveal}([\![v]\!], \mathcal{P})$: To reveal to the party $\mathcal{P}$ the value v. Every computation server transmits to the party its secret share, and the party reconstructs the value of v by summing the J secret shares.
- $[\![w]\!] \leftarrow \mathcal{F}_{MPC}.\texttt{Add}([\![u]\!], [\![v]\!])$: To compute the secret sharing of $w = u + v$ from the secret shared values of u and v. We also allow u or v to be in plaintext.
- $[\![w]\!] \leftarrow \mathcal{F}_{MPC}.\texttt{Mul}([\![u]\!], [\![v]\!])$: To compute the secret sharing of $w = uv$ from the secret shared values of u and v. As before, we allow u or v to be in plaintext.
- $r \leftarrow \mathcal{F}_{MPC}.\texttt{Rand}$: To generate a random value $r \in \mathbb{F}$. This functionality is called $J - 1$ times to secret-share a value v.

We extend the functionalities of $\mathcal{F}_{MPC}$ to allow u and v to not only have the form of finite field elements, but also vectors or matrices with compatible sizes.

With an additive MPC scheme, the secret sharing of a linear combination of values can be obtained by directly applying the linear combination to the corresponding secret shares of these values. Hence, the addition of two secretly shared values or the multiplication of a secretly shared value by a public value can be done without communication between the server. Examples of linear secret sharing schemes include SPDZ [18], BGW [19], and EMP [20].

We denote by Π the practical implementation of the MPC protocol used in this work. We remind that a neural network cannot be run directly through an MPC protocol due to its non-linearities, e.g. ReLU, division, Maxpool, and their derivatives. Instead, as a function can be approximated by a multinomial of the inputs, the approximation of the neural network can be run with Π.

C. Additive Homomorphic Encryption

A homomorphic encryption (HE) scheme allows computations to be performed directly on encrypted data without requiring decryption, enabling secure processing while ensuring that no information about the private inputs is revealed beyond what can be inferred from the final decrypted result. A more comprehensive discussion about HE can be found in [21].

In this work, we only consider the additive variant of homomorphic encryption (AHE). For a value v in the finite field $\mathbb{F}$, we define by $\langle v \rangle$ the homomorphic encryption of v, and assume that our AHE scheme $\mathcal{F}_{AHE}$ has the following functionalities.

- $(\texttt{pk}, \texttt{sk}) \leftarrow \mathcal{F}_{AHE}.\texttt{Setup}(\lambda)$: To generate the public and secret keys pk and sk with the security parameter λ. The public key is published and the secret key is kept by the key owner.
- $\langle x \rangle \leftarrow \mathcal{F}_{AHE}.\texttt{Enc}(x)$: To encrypt a plaintext x with the public key pk.
- $x \leftarrow \mathcal{F}_{AHE}.\texttt{Dec}(\langle x \rangle)$: To decrypt a ciphertext $\langle x \rangle$ to its plaintext value x. We assume that the ciphertext is calculated with the private key pk.
- $\langle w \rangle \leftarrow \mathcal{F}_{AHE}.\texttt{HAdd}(\langle u \rangle, v)$: To compute the ciphertext of $w = u + v$ with the encryption of u and the plaintext of v.
- $\langle w \rangle \leftarrow \mathcal{F}_{AHE}.\texttt{HCMul}(\langle u \rangle, v)$: To compute the ciphertext of $w = uv$ with the encryption of u and the plaintext of v.

With the functionalities listed above, a ciphertext can only be evaluated on a circuit of additions and multiplications on public constants with examples including the Paillier cryptosystem [22] and the additive El-Gamal cryptosystem [23]. A symmetric key variant to $\mathcal{F}_{AHE}$ can also be proposed, which is more efficient in general. In this case, $\mathcal{F}_{AHE}.\texttt{Setup}$ only generates the secret key sk, which is used for both encryption and decryption [24–26]. However, only the parties who know sk can encrypt,

and the other parties can only evaluate the ciphertext on an operation.

We denote by Φ the practical implementation of the HE protocol used in this work.

D. From MPC to HE

When using HE alone, it is sufficient to add a decrypting server to the set of parties. The decrypting server generates the public and secret keys p_k and s_k and then shares the public key with a party that owns x. This party can encrypt x with the public key and can decrypt $\langle x \rangle$ by sending $\langle y \rangle = \langle x \rangle + r$ to the decrypting server before recovering x from the plaintext y with $x = y - r$.

Encrypting with HE a secret shared value $[\![x]\!]$ is more complex. First, the computation servers run a MPC-aided joint key generation protocol to generate the public key $\mathtt{pk}$ and the secret-shared secret key $[\![\mathtt{sk}]\!]$ and then share $\mathtt{pk}$ with a party. Every computation server encrypts its secret-share before sending $\left\langle [\![x]\!]_j \right\rangle$ to this party. The party can recover $\langle x \rangle$ by summing the cipertexts of the secret shares, i.e. $\langle x \rangle = \sum_{j=1}^{J} \left\langle [\![x]\!]_j \right\rangle$. For decryption, the servers can use the same trick as with a decrypting server by jointly decrypting $\langle y \rangle = \langle x \rangle + r$ and then recovering x from the plaintext y. The privacy of the plaintext from the computation servers is guaranteed by the underlying MPC scheme, while the privacy of the secret key from the requester is again guaranteed by the security definition of the underlying HE scheme.

For this work, we denote the secure encryption and decryption of the underlying AHE scheme realised with masking or MPC by $\Phi.\mathtt{SecEnc}$ and $\Phi.\mathtt{SecDec}$.

E. Previous Works on Privacy-Enhanced VFL

Previous works have also considered the problem of privacy-enhanced VFL and proposed solutions to improve its runtime. However, these works either consider a different privacy definition or simpler assumptions for the global model.

The works of [27, 28] assume that the global model only aggregates the parties' feature representations, and thus has no parameters to optimise. [27] uses a secret-sharing based MPC scheme to allow for the aggregation to tolerate stragglers, while [28] uses homomorphic encryption scheme for the aggregation.

The work of [29] assumes a trainable global model, but only considers a single passive party. Privacy is obtained by adding and then removing noise from communicated values. However, this approach reveals the upper partial derivatives to the passive party, and some intermediary values in the forward propagation of the global model to the active party.

The works of [7, 11] treat the entire VFL network as one MPC function, at the cost of significantly higher complexity and slower runtime. In addition, [11] only works for two parties, and [7] can only be applied to the VFL setup if passive parties have no local model and use their data directly as their feature representations.

III. Framework

Optimization with VFL, Algorithm 1, is a distributed application of SGD, with the different model parameters updated with SGD at every optimization round. Each party needs to access the gradient of its model $\nabla_{\theta_i} \mathcal{L}(\mathcal{B})$ to perform this update. However, to execute, Algorithm 1 shows that the parties must share sensitive results. Specifically, it requires

1) Each passive party to transmit its feature representation $F_i[\mathcal{B}]$ to the active party.
2) The active party to transmit the upper partial derivatives $U_i[\mathcal{B}]$ to its associated passive party, or the passive party to transmit the lower partial derivatives $L_i[\mathcal{B}]$ to the active party.

In this work, we consider private the feature representations of the passive parties, their lower partial derivatives, and any intermediary result computed on top of them. Private results should remain hidden from every party, with the exception of the gradients of their local model needed for every party to update their parameters. While revealing the gradient can still expose the parties' data, we consider it to be an essential compromise needed for VFL to run. This privacy definition includes protecting the values of a party's gradient to the other parties, along with predictions, loss values, gradients for individual data points, and the elements needed for the chain rule calculation.

In Section III-A, we adapt VFL to protect all these quantities in the ideal world. In Section III-B, we present a baseline solution with MPC. In Section III-C, we present our solution, MUSE-VFL.

A. Ideal World

We consider the simulation-based security in which we assume the existence of a trusted third party $\mathcal{S}$ that helps the computation of the parties. In this simulation, we assume that each party will only receive the gradient

Algorithm 2 Privacy-Enhanced VFL - generic solution

1: *[Run Algorithm 1, replacing steps 5-11 with:]*
2: $\mathcal{P}_i$ encrypts $F_i[\mathcal{B}]$ and sends $\langle\langle F_i[\mathcal{B}] \rangle\rangle$ to $\mathcal{P}_0$.
3: $\mathcal{P}_0$ performs a forward pass with $\langle\langle F_i[\mathcal{B}] \rangle\rangle$, eq. (2).
4: $\mathcal{P}_0$ computes the batch loss $\langle\langle \mathcal{L}(\mathcal{B}) \rangle\rangle$.
5: $\mathcal{P}_0$ computes $\langle\langle \nabla_{\theta_0} \mathcal{L}(\mathcal{B}) \rangle\rangle$ and decrypts it.
6: $\mathcal{P}_0$ computes $\langle\langle U_i[\mathcal{B}] \rangle\rangle$, eq. (5).
7: $\mathcal{P}_i$ computes $L_i[\mathcal{B}]$, eq. (6).
8: $\langle\langle \nabla_{\theta_i} \mathcal{L}(\mathcal{B}) \rangle\rangle$ is calculated and revealed to $\mathcal{P}_i$

Note: $\mathcal{P}_0$ or $\mathcal{P}_i$ transmits either $\langle\langle U_i[\mathcal{B}] \rangle\rangle$ or $\langle\langle L_i[\mathcal{B}] \rangle\rangle$ to the other party to securely compute $\langle\langle \nabla_{\theta_i} \mathcal{L}(\mathcal{B}) \rangle\rangle$.

for their model parameters in each iteration, and no other value. We aim to realize such functionality through the use of privacy-enhancing technologies (PET), namely MPC and HE. Before we consider the concrete realization of the ideal functionality, first, we briefly discuss a generic realization of such functionality $\langle\langle\cdot\rangle\rangle$. In this generic realization, our focus is on the flow of a training iteration, and the actual PET being used will be discussed later. This generic solution can be found in Algorithm 2.

B. Generic solution through MPC

In Algorithm 3, we realise Algorithm 2 as a direct application of MPC. To simplify the notation, we denote the set of computation servers by $\mathcal{S}$, i.e. $\mathcal{S} = \{\mathcal{S}_1, \cdots \mathcal{S}_J\}$.

Algorithm 3 MPC-based Privacy-Enhanced VFL

1: *[Run Algorithm 1, replacing steps 5-11 with:]*
2: $\mathcal{P}_i$ secretly shares $F_i[\mathcal{B}]$ and sends $[\![F_i[\mathcal{B}]]\!]$ to $\mathcal{S}$.
3: $\mathcal{S}$ jointly calculate the batch loss $[\![\mathcal{L}(\mathcal{B})]\!]$.
4: $\mathcal{S}$ jointly calculate $[\![\nabla_{\theta_0}\mathcal{L}(\mathcal{B})]\!]$ and reveal it, $\Pi.\texttt{Reveal}([\![\nabla_{\theta_0}\mathcal{L}(\mathcal{B})]\!], \mathcal{P}_0)$.
5: $\mathcal{S}$ jointly calculate $[\![U_i[\mathcal{B}]]\!]$, eq. (5).
6: $\mathcal{P}_i$ computes $L_i[\mathcal{B}]$, Eq. (6).
7: $\mathcal{P}_i$ sends $[\![L_i[\mathcal{B}]]\!]$ to $\mathcal{S}$.
8: $\mathcal{S}$ jointly calculate $[\![\nabla_{\theta_i}\mathcal{L}(\mathcal{B})]\!]$.
9: $\mathcal{S}$ call $\Pi.\texttt{Reveal}([\![\nabla_{\theta_i}\mathcal{L}(\mathcal{B})]\!], \mathcal{P}_i)$.

As described in Section II-B, we consider $J \geq 2$ computation servers $\mathcal{S}_j$ in addition to the n passive parties P_i and the active party P_0. Without loss of generality, the active party is a computation server, and any passive party can be a computation server. If we consider that our MPC scheme has security guarantees with security parameter λ, then Algorithm 3 also benefits from the same security guarantees. Indeed, Algorithm 3 treats the forward pass on the global model, the backpropagation on the global model, and the one on the local model as a generic function that is then realized by the underlying MPC scheme.

We note that, when using MPC, the most efficient way for a passive party to compute its gradient is by secret sharing its lower partial derivatives with the computation servers as in Algorithm 3. Indeed, if the servers were to send their secret shares of the upper partial derivatives to the party, the party could reconstruct the plaintext of the upper partial derivatives, which would violate our privacy requirements. Alternatively, involving an additional party to send the secret shares to would still necessitate secret sharing the lower partial derivatives, making the efficiency even worse.

C. MUSE-VFL

With Algorithm 3, every passive party secret shares with the computation servers their lower partial derivatives to allow the calculation of equation (4). However, this calculation requires the multiplication between a matrix in $U_i[\mathcal{B}]$ with dimension $(1, d_i)$ and a matrix in $L_i[\mathcal{B}]$ with dimension $(d_i, |\theta_i|)$. Considering that an upper partial derivative has significantly fewer values than a lower partial derivative, d_i against $d_i|\theta_i|$ values, the computation servers should privately pass down the upper partial derivatives to the passive parties to reduce the communication bandwidth. To this end, we propose in Algorithm 4 for each computation server to encrypt its secret share $[\![U_i[\mathcal{B}]]\!]_j$ with a homomorphic encryption protocol $\langle\cdot\rangle$ and then to transmit $\left\langle [\![U_i[\mathcal{B}]]\!]_j \right\rangle$ to its associated passive party, which recovers the ciphertext of the upper partial derivative by summing the shares of every party, $\langle U_i[\mathcal{B}]\rangle = \sum_{j=1}^{J} \left\langle [\![U_i[\mathcal{B}]]\!]_j \right\rangle$. We present this solution in Algorithm 4 and further compare its complexity to that of Algorithm 3 in Section IV.

Algorithm 4 Our solution MUSE-VFL

1: *[Run Algorithm 3, replacing steps 7-9 with:]*
2: $\{\mathcal{S}_j\}_{j=1}^{J}$ calls $\langle U_i[\mathcal{B}]\rangle \leftarrow \Phi.\texttt{SecEnc}([\![U_i[\mathcal{B}]]\!], \mathcal{P}_i)$.
3: $\mathcal{P}_i$ computes $\langle \nabla_{\theta_i}\mathcal{L}(\mathcal{B})\rangle$ with $\langle U_i[\mathcal{B}]\rangle$ and $L_i[\mathcal{B}]$.
4: $\mathcal{P}_i$ decrypts $\nabla_{\theta_i}\mathcal{L}(\mathcal{B}) \leftarrow \Phi.\texttt{SecDec}(\langle \nabla_{\theta_i}\mathcal{L}(\mathcal{B})\rangle)$.

In Algorithm 4, the security against the computation servers is guaranteed by the security guarantee of the underlying MPC. Furthermore, the computation servers encrypt the secret shares of $U_i[\mathcal{B}]$ with homomorphic encryption before transmitting them to the passive party $\mathcal{P}_i$. This implies that $\mathcal{P}_i$ receives a ciphertext for each element in $U_i[B]$. Hence, from the passive parties' point of view, the privacy of the other parties' private data is guaranteed by the security guarantee of the underlying homomorphic encryption. Lastly, the security of the decryption step is guaranteed by that of $\Phi.\texttt{SecDec}$.

For symmetrical AHE, since ciphertexts are never sent to the computation servers, each server can have access to the secret key and directly perform encryption. This prevents the need for an encrypting/decrypting server. Moreover, the masking scheme described in Section II-D enables decryption of the local gradient with a single computation server instead of a joint decryption scheme or a dedicated decryption server.

IV. Complexity Analysis

In this section, we look at the complexity of the MPC baseline, Algorithm 3, and MUSE-VFL, Algorithm 4. These two algorithms only differ in their computation of the gradients for the passive parties. We analyse how this step affects the complexity of a passive party in Section IV-A, a computation server in Section IV-B, and extend these analyses to other types of parties in Section IV-C.

The baseline and MUSE-VFL only differ on lines 7-9 for Algorithm 3 and lines 2-4 for Algorithm 4. We refer

to these steps respectively by f_{MPC} and f_{MUSE}. Moreover, we use $C(f)$ to denote the complexity of a function f, and $C(f, X)$ to represent the complexity of the steps in the function f involving party X. When expressing the complexity of MUSE-VFL and the baseline, we assume that any field element addition is negligible compared to the field multiplication and other building blocks.

In addition to using Π to refer to the MPC protocol and Φ for the HE protocol, we use Ψ to refer to the standard sending protocol, and $\Psi.\texttt{Send}$ to refer to sending a value. We assume that sending a value only incurs a cost to the sender and no cost to the receiver.

A. Complexity of a Passive Party

Let us consider a passive party $\mathcal{P}_i$. The computation servers compute the upper partial derivatives $U_i[\mathcal{B}]$ in both algorithms. However, the passive party secretly shares the lower partial derivatives $L_i[\mathcal{B}]$ with the computation servers in the baseline, while the computation servers transmit the ciphertext of their secret share of $U_i[\mathcal{B}]$ to the passive party in MUSE-VFL.

MPC baseline. $U_i[\mathcal{B}]$ is the set of B upper partial derivatives. Each partial derivative is a matrix of dimension $(d_i, |\theta_i|)$ and thus requires to secret share $d_i|\theta_i|$ values, where d_i is the dimension of the output of the local model. By summing over every element in the batch, we have

$$C(f_{MPC}, \mathcal{P}_i) = Bd_i|\theta_i|[(J-1)C(\Pi.\texttt{Rand}) + JC(\Psi.\texttt{Send})]\,.$$

MUSE-VFL. $U_i[\mathcal{B}]$ is the set of B upper partial derivatives. Each upper partial derivative is a matrix of dimension $(1, d_i)$ and the reconstruction of its ciphertext, line 2, requires to add the $d_i(J-1)$ ciphertexts of its shares. Furthermore, for a data sample, computing the homomorphic encryption of its local gradient requires the matrix multiplication of $\langle U_i[\mathcal{B}]\rangle$ and $L_i[\mathcal{B}]$, and thus requires $|\theta_i|d_i$ homomorphic constant multiplications and $|\theta_i|(d_i - 1)$ homomorphic additions. The ciphertext of the gradient is then obtained by summing the B ciphertexts of the individual gradient for every sample in the batch, which requires $(B-1)|\theta_i|$ homomorphic additions. Finally, the passive party decrypts it by applying $\Phi.\texttt{SecDec}$ to every parameter.

If we assume that the decryption is done by an external decrypting server, the passive party generates and adds a random number to mask each of its parameters before sending its masked values to the decrypting server. Thus, the complexity of $\Phi.\texttt{SecDec}$ for a single value can be further decomposed as

$$C(\Phi.\texttt{SecDec}) = C(\Pi.\texttt{Rand}) + C(\Phi.\texttt{HAdd}) + C(\Psi.\texttt{Send}),$$

and

$$\begin{aligned} C(f_{MUSE}, \mathcal{P}_i) =& |\theta_i|[C(\Pi.\texttt{Rand}) + C(\Psi.\texttt{Send}] \\ &+ Bd_i|\theta_i|C(\Phi.\texttt{HCMul}) \\ &+ Bd_i\,[(J-1) + |\theta_i|]\,C(\Phi.\texttt{HAdd}). \end{aligned}$$

MPC baseline vs MUSE-VFL. With the baseline, the complexity of a passive party is only the one needed to secret-share its lower partial derivatives. Instead, MUSE-VFL reduces the number of random values generated and sent from $\Theta(Bd_i|\theta_i|J)$ to $\Theta(|\theta_i|)$. However, this improvement comes at the cost of $\Theta(Bd_i|\theta_i|)$ additional homomorphic operations for MUSE-VFL instead of the generation of $\Theta(Bd_i|\theta_i|J)$ random values. It is reasonable to assume that randomly generating a number, $\Pi.\texttt{Rand}$, has similar complexity to adding or multiplying ciphertext with plaintext values, $\texttt{HCAdd}$ and $\texttt{HMul}$. Hence, asymptotically, the complexity of MUSE-VFL is lower than that of the MPC baseline for every passive party.

B. Complexity of a Computation Server

Let us consider the computation server $\mathcal{S}_j$. The computation servers compute, for every passive party, the set of upper partial derivatives $U_i[\mathcal{B}]$ in both algorithms. However, the computation servers compute the local gradient of a passive party with $[\![L_i[\mathcal{B}]]\!]$ and $[\![U_i[\mathcal{B}]]\!]$ in the baseline, while every computation server transmits a homomorphic encryption of its secret share $[\![U_i[\mathcal{B}]]\!]_j$ to the passive party for it to compute the gradient with $\langle U_i[\mathcal{B}]\rangle$ in MUSE-VFL.

MPC baseline. Let us consider a passive party $\mathcal{P}_i$ that transmits to the computation server a secret share for each of its elements in $L_i[\mathcal{B}]$. The computation servers perform a secure multiplication between $[\![U_i[\mathcal{B}]]\!]$ and $[\![L_i[\mathcal{B}]]\!]$, each with shares of respective dimension $(1, d_i)$ and $(d_i, |\theta_i|)$, to compute the secret share of the local gradient. This requires $d_i|\theta_i|$ multiplications for each passive party and data sample. The computation server then sums the gradient of each sample in the batch and sends to each passive party a share of the batch gradient,

$$\begin{aligned} C(f_{MPC}, \mathcal{S}_j) =& B\left(\sum_{i=1}^{n} d_i|\theta_i|\right)[C(\Pi.\texttt{Mul})] \\ &+ B\left(\sum_{i=1}^{n} |\theta_i|\right)C(\Psi.\texttt{Send}). \end{aligned}$$

MUSE-VFL. The computation server homomorphically encrypts its secret share of every element in $U_i[\mathcal{B}]$, each of dimension $(1, d_i)$, before sending them to the passive party. The computation server needs to do this for every passive party and data sample in the batch. Hence, we get

$$C(f_{MUSE}, \mathcal{S}_j) = B\left(\sum_{i=1}^{n} d_i\right)[C(\Phi.\texttt{Enc}) + C(\Psi.\texttt{Send})]\,.$$

MPC baseline vs MUSE-VFL. First, regarding the communication complexity, we remind that a secure MPC-based multiplication between two secret shared values requires $\Omega(J)$ communications for each value [30]. Therefore, MUSE-VFL reduces the number of communications from $\Omega(B\sum_{i=1}^{n}|\theta_i|Jd_i)$ to $B\sum_{i=1}^{n}d_i$. Regarding the computation complexity, the computation servers perform $B\sum_{i=1}^{n}d_i|\theta_i|$ secure MPC multiplications with the baseline, while performing instead $B\sum_{i=1}^{n}d_i$ encryptions with MUSE-VFL. A secure MPC multiplication requires the generation of correlated random values [18, 19] for the Beaver triple [18, 20], and thus the use of somewhat homomorphic encryption or oblivious transfer. Therefore, the complexity of a secure MPC multiplication is superior to that of an HE encryption, and the complexity of MUSE-VFL is lower than that of the MPC baseline for every computation server.

C. Complexity of Other Parties

Active party. The active party owns the global model and also plays the role of a computation server. For both MUSE-VFL and the baseline, the global gradient is computed across the computation servers and then revealed to the active party. MUSE-VFL does not change this step, which is still done with MPC. Hence, the active party has the complexity of a computation server in our analysis of the computation of the local gradients for both MUSE-VFL and the baseline.

A passive party that is also a computation server. This case is particularly important as it provides the passive party a way to be involved in the secure computation while also avoiding the involvement of external parties as computation servers. In this case, the complexity of the passive party is almost equal to the former one for the passive party and the one for the computation server. The only difference is that every passive transmits one fewer share to the other parties for each of its value, which is asymptotically negligible.

V. Experimental Analysis

In this section, we present our experimental results across four distinct machine learning tasks, with each task evaluating a different model architecture and dataset. For each of them, we show the improvements of MUSE-VFL over the MPC baseline. Our source code is made publicly available at https://github.com/muse-public-repo/MUSE-VFL.

A. System Setting

We evaluate protocols on a machine equipped with 24 Gen Intel Core i9-12900K processors @ 5.2GHz and 1 TB of RAM, running Ubuntu 22.04.2 LTS. The implementation is done in C++ with HEXL[1] for HE-based computation, EMP-toolkit[2] for two-party computation, and OpenSSL[3] to support our use of HEXL and EMP-toolkit. All experiments simulate two participants on a single machine with in-memory storage, under both a simulated LAN setting with an average bandwidth of 625 MB/s and a simulated WAN setting with an average bandwidth of 100 MB/s and a 40 ms delay. Each participant acts as both a passive party and computation server, with one participant additionally serving as an active party. In our experiments, data is represented using a 32-bit fixed-point format, with 16 bits, including the sign bit, allocated to the integer part and 16 bits to the fractional part. We set the security level to 80 bits for both the HE and MPC implementations to ensure a fair comparison, specifically with a 1024-bit modulus for Paillier and a 1024-degree polynomial for BFV.

B. Datasets and Models

We assess the performance of MUSE-VFL on four tasks. For each of them, 80% of the original dataset is allocated for training and the rest for testing.

MNIST for image classification. As in the work of [31], we vertically split each image of dimension 1×28×28 between the two parties into two subimages of dimension 1×28×14. Each party's local model is composed of two 3×3 convolutional layers with 64 channels, followed by a fully connected layer with 256 units. The global model concatenates the feature representations of the two parties and applies logistic regression for multi-class classification.

Avazu for click-through rate prediction. As in the work of [32], we partition the features between the two parties into 8 and 14 features, respectively. Each party applies an embedding layer for feature encoding, followed by two fully connected layers. The global model then combines the outputs using a dot product, followed by a sigmoid for prediction.

Credit for credit risk assessment. As in the work of [33], we partition the features between the two parties into 11 and 12 features, respectively. Each party's local model is composed of three fully connected layers. The global model runs a logistic regression on the concatenated output of the two parties.

BAF [34] for bank fraud detection. We one-hot encode categorical variables, thus increasing the number of features from 31 to 47, and use SMOTE to balance the training set as in [35]. We then partition the features between the two parties into 24 and 23 features, respectively. We adopt a model architecture similar to that used for the

[1]https://github.com/intel/hexl
[2]https://github.com/emp-toolkit
[3]https://openssl.org

TABLE I
ACCURACY FOR MNIST, AVAZU, AND CREDIT, AND TPR FOR BAF.

Dataset	Model Performance (in %)		
	Plaintext	MPC baseline	MUSE-VFL
MNIST	95.34	95.28	94.89
Avazu	82.30	82.21	82.03
Credit	81.40	81.32	80.78
BAF	29.09	28.89	28.42

Avazu dataset, but replace the dot product operation with two fully connected layers.

C. Results and Discussions

Now, we present the experimental results. We show that MUSE-VFL maintains accuracy, while significantly reducing total runtime.

Accuracy Preservation. Table I presents the model performance across the four ML tasks for three VFL solutions: plaintext, the MPC baseline, and MUSE-VFL with BFV. We report the accuracy for MNIST, Avazu, and Credit, and the TPR for BAF due to its imbalanced nature. More than 98% of BAF's data samples are of the same class. For each task, we consider a learning rate and number of epochs equal to 0.001 and 95 for MNIST, 0.01 and 10 for Avazu, 0.0005 and 18 for Credit, and 0.001 and 28 for BAF. For every task, we also consider batches of size 32. We note that the accuracy of the MPC baseline is slightly lower than that of the plaintext version, with a decrease ranging from 0.06% to 0.09%, due to the use of approximation techniques for MPC-friendly computation. MUSE-VFL shows a slightly larger model accuracy drop compared to the plaintext-based solution, ranging from 0.27% to 0.62%, which is attributed to the additional approximations introduced by the encryption scheme used for HE. Overall, both cryptographic solutions achieve model performance comparable to the plaintext baseline, and the observed variations are consistent with the protocol design.

Runtime breakdown for gradient computation. Table II breaks down the time needed for MUSE-VFL and the baseline to compute the gradient of a single data sample. For MUSE-VFL, this includes the MPC-to-HE conversion process, which involves encrypting and sending the shares in ciphertext, as well as computation over HE ciphertext. For the baseline, this includes the passive party secret sharing its lower partial derivatives, the multiplication of the lower and upper partial derivatives by the computation servers, and sending the secret shares of the gradient to the passive party. We categorize the time for these steps under encryption, communication, and computation to reflect the trade-offs outlined in Section IV. We observe that both the Paillier-based and BFV-based versions MUSE-VFL improve significantly the runtime of the baseline, approximately 32% to 37% faster for LAN and 32% to 52% for WAN, respectively, consistently across all four ML tasks. We also observe that communication is the main factor in the runtime of the baseline, and that MUSE-VFL reduces the communication time, at the cost of a fixed increase in computation and encryption time. In the rest of this work, we only consider the BFV-based MUSE-VFL as it outperforms Pailler-based MUSE-VFL on every task.

TABLE II
BREAKDOWN OF THE RUNTIME FOR ENCRYPTION, COMPUTATION, AND COMMUNICATION FOR THE LOCAL BACKPROPAGATION OF A SINGLE DATA SAMPLE UNDER LAN AND WAN SETTINGS, COMPARING THE BASELINE (MPC) AND OUR SOLUTION (PAILLIER AND BFV).

Dataset	Method	Enc.	Comp.	Comm.		Total	
				LAN	WAN	LAN	WAN
MNIST	MPC	–[a]	0.05	273	7234[c]	273	7234[c]
	Pailler	177[b]	5.87	0.31	4617	183	4800
	BFV	168	3.36	0.27	4018	172	4190
Avazu	MPC	–	0.09	294	7597	293	7597
	Paillier	212	6.80	0.35	5192	219	5412
	BFV	194	4.30	0.34	4961	199	5159
Credit	MPC	–	0.02	200	5194	200	5194
	Paillier	128	4.32	0.21	3184	133	3317
	BFV	121	2.75	0.21	3111	125	3234
BAF	MPC	–	0.06	236	6196	236	6196
	Paillier	162	4.08	0.20	3602	166	3768
	BFV	151	2.46	0.19	2850	152	3003

[a] We use '–' if not applicable.
[b] Every time in this table is in ms.
[c] The value for Communication and Total may match when Computation is negligible due to value rounding.

Runtime breakdown for one SGD. Table III breaks down the time and communication volume of a single SGD step for a batch size of 32. The time of a SGD step is primarily driven by the inference and backpropagation runtime of the local model, as the VFL model contains far more local parameters than global ones. Furthermore, the local backpropagation runtime substantially outweighs the combined runtime of all other steps, primarily due to the use of cryptographic tools to compute local gradients. This highlights the critical importance of this work to optimize this specific time. Also, Table III shows that MUSE-VFL has a communication volume slightly smaller than the baseline by only around 3%. Indeed, encrypting the secret shares increases their size to 1024 bits ciphertexts for 80 bits security. However, MUSE-VFL remains faster by requiring fewer communication rounds. In MUSE-VFL, parties only communicate to reconstruct the encryption of the upper partial derivatives and decrypt their gradient, while, in the baseline, they communicate to secret share their lower partial derivatives, but also to securely multiply their partial derivatives and send back the gradient.

Summary of VFL training. Table IV presents the total runtime and communication volume for MUSE-VFL and the baseline, measured for one SGD step and the entire

TABLE III

BREAKDOWN OF THE RUNTIME AND COMMUNICATION VOLUME OF ONE SGD STEP UNDER LAN AND WAN SETTINGS: LOCAL INFERENCE (L-I), GLOBAL INFERENCE (G-I), GLOBAL BACKPROPAGATION (G-B), AND LOCAL BACKPROPAGATION (L-B).

Dataset	Method	Comm. Vol. (in KB)				LAN - Runtime (in ms)				WAN - Runtime (in s)			
		L-I	G-I	G-B	L-B	L-I	G-I	G-B	L-B	L-I	G-I	G-B	L-B
MNIST	MPC	–	185	183	2738	561	0.30	0.30	8756	0.56	0.03	0.03	232
	Ours				2645				5531				134
Avazu	MPC	–	186	184	3832	585	0.21	0.21	9398	0.59	0.02	0.02	243
	Ours				3712				6381				165
Credit	MPC	–	95	95	2934	810	0.30	0.30	6401	0.81	0.02	0.02	166
	Ours				2853				3990				104
BAF	MPC	–	198	195	3172	580	25	26	7541	0.58	2.27	2.27	198
	Ours				3043				4902				96

TABLE IV

SUMMARY OF THE COMMUNICATION VOLUME AND TIME REQUIRED OVER LAN AND WAN TO PERFORM ONE SGD STEP OR COMPLETE THE ENTIRE VFL TRAINING PROCESS.

Dataset	Method	Comm.[a]		LAN[b]		WAN[b]	
		Batch	Total	Batch	Total	Batch	Total
MNIST	Baseline	3.11	541	9.32	461	232	11486
	Ours	3.01	524	6.09	301	135	6665
Avazu	Baseline	4.20	115	9.98	77.9	244	1904
	Ours	4.08	112	6.97	54.4	166	1295
Credit	Baseline	3.12	46.3	7.21	30.4	167	705
	Ours	3.06	45.4	4.80	20.3	104	440
BAF	Baseline	3.56	85.1	8.08	53.1	203	1335
	Ours	3.44	82.2	5.53	37.5	101	664

[a] Communications are in MB for Batch and GB for Total.
[b] Runtimes are in seconds for Batch and in hours for Total.

training. We use the same hyperparameters as for Table I. For a single batch, we report the average communication volume and time measured over five batches, then extrapolate it to estimate the overall performance of the entire VFL training based on the total number of SGD steps needed for convergence. Although MUSE-VFL involves an additional step for the passive party to decrypt its gradient, the overall communication cost is still lower than that of the baseline, which requires secret sharing the lower partial derivatives and securely multiplying the lower and upper partial derivatives. We observe a total runtime improvement of approximately 30% to 35% for LAN across the four ML tasks and 32% to 50% for WAN, with an affordable compromise in model performance as shown earlier. This highlights the tradeoff between training effectiveness and execution efficiency.

VI. CONCLUSION

In this work, we considered the problem of privacy-enhanced VFL, in which every passive party owns a local model and only accesses its parameters in plaintext across the VFL training. We introduced a baseline that uses MPC to solve this problem. Every passive party secret shares its feature representations and lower partial derivatives to a set of computation servers. In return, it receives its gradient, ensuring that neither the computation servers nor any other party gains access to the underlying computations performed to derive the gradient. All our experiments demonstrated that the MPC baseline runtime is driven by the time needed to compute every party's gradient. This computation takes significantly more time than the rest of the training. Hence, we also proposed MUSE-VFL, which computes the parties' gradients faster. Rather than having the passive parties send their lower partial derivatives to the computation servers, they receive a homomorphic encryption of the upper partial derivatives that they use to compute the ciphertext of the gradient. By using HE and moving the gradient computation from the computation servers to the passive parties, fewer values are transmitted, making MUSE-VFL faster than the MPC baseline. We showed, theoretically, how MUSE-VFL improves the complexity of this step, and, experimentally, that it outperforms the training time of the baseline by 30% to 35% for LAN and 32% to 50% for WAN.

ACKNOWLEDGEMENT

This work is a result of the joint lab collaboration between Ant International and Nanyang Technological University, established as part of a five-year partnership to advance digital trust and privacy-enhancing technologies (PETs), with funding provided by Ant International. This research is supported by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Infocomm Media Development Authority

REFERENCES

[1] Y. Cheng, Y. Liu, T. Chen, and Q. Yang, "Federated learning for privacy-preserving ai," *Commun. ACM*, vol. 63, p. 33–36, Nov. 2020.

[2] G. Long, Y. Tan, J. Jiang, and C. Zhang, "Federated learning for open banking," in *Federated learning: privacy and incentive*, pp. 240–254, Springer, 2020.

[3] C.-j. Huang, L. Wang, and X. Han, "Vertical federated knowledge transfer via representation distillation for healthcare collaboration networks," in *Proceedings of the ACM Web Conference*, 2023.

[4] Y. Hu, D. Niu, J. Yang, and S. Zhou, "Fdml: A collaborative machine learning framework for distributed features," in *ACM SIGKDD*, 2019.

[5] Y. Liu, Y. Kang, C. Xing, T. Chen, and Q. Yang, "A secure federated transfer learning framework," *IEEE Intelligent Systems*, vol. 35, no. 4, 2020.

[6] C. Fu, X. Zhang, S. Ji, J. Chen, J. Wu, S. Guo, J. Zhou, A. X. Liu, and T. Wang, "Label inference attacks against vertical federated learning," in *31st USENIX security symposium*, 2022.

[7] Y. Wu, N. Xing, G. Chen, T. T. A. Dinh, Z. Luo, B. C. Ooi, X. Xiao, and M. Zhang, "Falcon: A privacy-preserving and interpretable vertical federated learning system," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2471–2484, 2023.

[8] Y. Liu, Y. Kang, T. Zou, Y. Pu, Y. He, X. Ye, Y. Ouyang, Y.-Q. Zhang, and Q. Yang, "Vertical federated learning: Concepts, advances, and challenges," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, 2024.

[9] M. Ye, W. Shen, B. Du, E. Snezhko, V. Kovalev, and P. C. Yuen, "Vertical federated learning for effectiveness, security, applicability: A survey," *ACM Computation Survey*, vol. 57, Apr. 2025.

[10] P. Mohassel and Y. Zhang, "Secureml: A system for scalable privacy-preserving machine learning," in *IEEE symposium on security and privacy*, 2017.

[11] S. Wagh, D. Gupta, and N. Chandran, "Securenn: 3-party secure computation for neural network training," *Proceedings on Privacy Enhancing Technologies*, vol. 2019, pp. 26–49, 07 2019.

[12] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, "Cheetah: Optimizing and accelerating homomorphic encryption for private inference," in *IEEE International Symposium on HPCA*, 2021.

[13] M. Ramadasan, O. Tahmi, H. Ould-Slimane, C. Talhi, and G. Suganya, "Privacy-preserving machine learning inference for intrusion detection," in *International Symposium on Foundations and Practice of Security*, pp. 29–42, Springer, 2025.

[14] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC conference*, pp. 308–318, 2016.

[15] A. Triastcyn and B. Faltings, "Bayesian differential privacy for machine learning," in *International Conference on Machine Learning*, PMLR, 2020.

[16] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: A low latency framework for secure neural network inference," in *27th USENIX security symposium*, pp. 1651–1669, 2018.

[17] R. Cramer, I. B. Damgård, and J. B. Nielsen, *Secure multiparty computation and secret sharing*. Cambridge University Press, 2015.

[18] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Annual Cryptology Conference*, pp. 643–662, Springer, 2012.

[19] M. Ben-Or, S. Goldwasser, and A. Wigderson, "Completeness theorems for non-cryptographic fault-tolerant distributed computation," in *20th ACM symposium on Theory of computing*, 1988.

[20] X. Wang, S. Ranellucci, and J. Katz, "Authenticated garbling and efficient maliciously secure two-party computation," in *Proceedings of the 2017 ACM SIGSAC conference*, pp. 21–37, 2017.

[21] C. Gentry, *A fully homomorphic encryption scheme*. Stanford university, 2009.

[22] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Advances in Cryptology — EUROCRYPT '99*, Springer, 1999.

[23] R. Cramer, R. Gennaro, and B. Schoenmakers, "A secure and optimally efficient multi-authority election scheme," *Advances in Cryptology — EUROCRYPT '97*, pp. 103–118, 1997.

[24] A. Blum, M. Furst, M. Kearns, and R. J. Lipton, "Cryptographic primitives based on hard learning problems," in *Advances in Cryptology — CRYPTO' 93*, pp. 278–291, Springer, 1993.

[25] B. Applebaum, D. Cash, C. Peikert, and A. Sahai, "Fast cryptographic primitives and circular-secure encryption based on hard learning problems," in *Advances in Cryptology - CRYPTO 2009*, pp. 595–618, Springer, 2009.

[26] L. Ducas and D. Micciancio, "Fhew: bootstrapping homomorphic encryption in less than a second," in *Annual international conference on the theory and applications of cryptographic techniques*, pp. 617–640, Springer, 2015.

[27] S. Li, D. Yao, and J. Liu, "Fedvs: Straggler-resilient and privacy-preserving vertical federated learning for split models," in *ICML*, PMLR, 2023.

[28] C. G. Allaart, M. X. Makkes, L. Dijksman, P. van der Nat, D. Biesma, H. Bal, and A. v. Halteren, "Secure and private vertical federated learning for predicting personalized cva outcomes," in *Artificial Intelligence in Medicine*, (Cham), pp. 172–181, Springer Nature Switzerland, 2024.

[29] Y. Zhang and H. Zhu, "Additively homomorphical encryption based deep neural network for asymmetrically collaborative machine learning," *arXiv preprint arXiv:2007.06849*, 2020.

[30] I. Damgård, K. G. Larsen, and J. B. Nielsen, "Com-

munication lower bounds for statistically secure mpc, with or without preprocessing," in *Advances in Cryptology – CRYPTO 2019*, Springer, 2019.

[31] Y. Liu, X. Zhang, Y. Kang, L. Li, T. Chen, M. Hong, and Q. Yang, "FedBCD: A communication-efficient collaborative learning framework for distributed features," *IEEE Transactions on Signal Processing*, vol. 70, pp. 4277–4290, 2022.

[32] F. Fu, X. Miao, J. Jiang, H. Xue, and B. Cui, "Towards communication-efficient vertical federated learning training via cache-enabled local updates," *Proc. VLDB Endow.*, vol. 15, no. 10, 2022.

[33] T. Zou, Z. Gu, Y. He, H. Takahashi, Y. Liu, and Y. Zhang, "VFLAIR: A research library and benchmark for vertical federated learning," in *ICLR*, 2024.

[34] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, R. P. Ribeiro, J. Gama, and P. Bizarro, "Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation," in *NeurIPS*, 2022.

[35] T. Awosika, R. M. Shukla, and B. Pranggono, "Transparency and privacy: the role of explainable ai and federated learning in financial fraud detection," *IEEE Access*, 2024.